EXPLORATORY DATA ANALYSIS

Chapter 19

Time Series EDA

Understanding data whose order in time matters

Course tools: Python, NumPy, Pandas, Matplotlib's object-oriented API, and Seaborn.

This chapter focuses on temporal structure and descriptive investigation—not forecasting or predictive time-series models.

Learning objectives

• Recognize a time series as observations indexed by time and identify its temporal grain.

• Distinguish event time, observation time, ingestion time, and processing time.

• Audit timestamps, time zones, ordering, duplicate times, irregular intervals, and missing periods.

• Explore trend, seasonality, cycles, calendar effects, and structural changes.

• Understand lag, autocovariance, autocorrelation, and their limitations.

• Inspect rolling summaries, changing variance, temporal gaps, and unusual periods.

• Choose aggregation frequency carefully and avoid misleading comparisons.

• Build a reproducible time-series EDA workflow with Pandas and Matplotlib's object-oriented API.

Central principle: time series are not ordinary tables with a date column. Their order, spacing, calendar, and data-collection process are part of the meaning of the observations.

1. What is a time series?

A time series is a sequence of observations indexed by time. We can write it as {(t■, y■), (t■, y■), …, (t■, y■)}, where t■ is a timestamp and y■ is the measured value. The sequence may be regularly sampled (for example, every hour) or irregularly sampled (for example, whenever a customer makes a purchase).

A time series can be univariate, with one measured variable, or multivariate, with several variables observed over time. Examples include daily sales, hourly temperature, monthly unemployment, sensor readings, website traffic, and customer events.

Concept	Meaning	Example
Temporal index	When the observation applies	2026-10-01 09:00
Value	What was measured	Temperature = 28.4 °C
Frequency / cadence	Expected spacing between observations	Hourly
Temporal grain	The time interval represented by a row	One sensor reading per minute

The row's temporal grain must be explicit. A row may represent an instant, a duration, an event, or an aggregate over a period. Confusing these interpretations creates errors in analysis.

2. Review of earlier chapters (Chapters 1–17)

This review excludes Chapter 18, which is reviewed separately in the next section.

• **Ch. 1 — EDA:** use questions and context; observation is not explanation.

• **Ch. 2 — Dataset understanding:** define observational unit, grain, variable meaning, measurement scale, and data-generating process.

• **Ch. 3 — Data quality:** check completeness, validity, consistency, uniqueness, accuracy, and timeliness.

• **Ch. 4 — Numerical variables:** use quantiles, spread, robust summaries, and distribution shape.

• **Ch. 5 — Categorical variables:** inspect frequencies, rare categories, cardinality, and denominators.

• **Ch. 6 — Date/time:** parse timestamps, handle time zones, inspect calendar fields and event intervals.

• **Ch. 7 — Descriptive statistics:** select summaries according to distribution and purpose.

• **Ch. 8 — Distributions:** understand empirical distributions, tails, gaps, and density displays.

• **Ch. 9 — Shape:** skewness, kurtosis, and quantiles describe distributional shape, not validity.

• **Ch. 10 — Distribution visualization:** use histograms, ECDFs, boxplots, and related displays.

• **Ch. 11 — Relationships:** inspect form, strength, nonlinearity, overplotting, and confounding.

• **Ch. 12 — Multivariate visualization:** use facets, heatmaps, and encodings with care.

• **Ch. 13 — Association:** choose measures for variable types; association is not causation.

• **Ch. 14 — Covariance and dependence:** inspect joint and conditional structure, not only marginal summaries.

• **Ch. 15 — Grouped analysis:** compare groups with explicit denominators and aggregation choices.

• **Ch. 16 — Outlier detection:** flags are method-dependent candidates, not automatic deletion instructions.

• **Ch. 17 — Investigating unusual observations:** trace source, compare peers, quantify influence, and document decisions.

3. Review of Chapter 18 — High-Dimensional EDA

Chapter 18 introduced variable inventories, domain-based column groups, missingness across rows and columns, constants and near-constants, duplicate and redundant fields, pairwise associations, correlation heatmaps, mixed-type relationships, and the risks of multiple comparisons.

> **Expected reasoning for the Chapter 18 assignment/project:** a good audit explains the observational unit, documents variables, preserves original columns, reports missingness with denominators, selects relationships deliberately, includes pairwise sample sizes, and avoids causal claims from exploratory association. If no assignment was submitted, this is a reasoning guide rather than a grading result.

Time series add an important dimension: the order and spacing of observations. A dataset may have valid values but still be analytically misleading because periods are missing, timestamps are misaligned, or aggregation hides temporal variation.

4. Define the time semantics before analysis

4.1 Event time, observation time, and ingestion time

Time concept	Meaning	Example
Event time	When the real-world event happened	A purchase occurred at 10:04
Observation time	When a sensor or system recorded it	Sensor sampled at 10:05
Ingestion time	When data entered the platform	Record arrived at 10:09
Processing time	When a pipeline processed it	Batch ran at 10:15

These timestamps answer different questions. Late-arriving events can make ingestion-time charts look different from event-time charts. Store the relevant timestamps separately when possible.

4.2 Temporal grain

Clarify whether each row is an event, an instantaneous measurement, or an aggregate over an interval. For example, a row labelled 2026-10-01 may represent a calendar-day total, a midnight snapshot, or a measurement taken at an unspecified time. These are not interchangeable.

4.3 Time zones and daylight saving

A timestamp without a time-zone interpretation may be ambiguous. Normalize instants to a consistent timezone (often UTC) for ordering and elapsed-time calculations, while retaining local time when local calendar behavior matters. A local day can be 23 or 25 hours in regions observing daylight saving time.

```
# Parse timestamps and inspect failures.
df["timestamp"] = pd.to_datetime(
    df["timestamp"], errors="coerce", utc=True
)
parse_failures = df.loc[df["timestamp"].isna()].copy()

# If source strings are known to be local time, localize explicitly
# before converting to UTC; do not assume they are UTC.
```

5. Establish ordering, uniqueness, and cadence

Sort by time before calculating differences, rolling summaries, or lags. For panel data, sort within each entity. Check duplicate timestamps and decide whether duplicates represent repeated measurements, multiple events, or accidental duplication.

```
df = df.sort_values(["entity_id", "timestamp"]).copy()

# Inspect duplicate entity-time combinations.
duplicate_time_rows = df.loc[
    df.duplicated(["entity_id", "timestamp"], keep=False)
].sort_values(["entity_id", "timestamp"])

# Time gaps within each entity.
df["time_gap"] = (
    df.groupby("entity_id")["timestamp"].diff()
)
```

Do not automatically drop duplicate timestamps. Two valid events can occur at the same timestamp resolution. If multiple measurements should be combined, choose an aggregation rule based on the measurement meaning.

5.1 Regular versus irregular sampling

Regular series have an expected cadence, such as one observation every 15 minutes. Irregular series are event-driven or have variable intervals. An irregular series is not necessarily defective; the expected sampling mechanism determines whether gaps are meaningful.

```
# Inspect observed gaps for a single series.
one = (
    df.loc[df["entity_id"].eq("A")]
      .sort_values("timestamp")
      .set_index("timestamp")
)
gaps = one.index.to_series().diff().value_counts()
gaps.head(10)
```

6. Temporal coverage and missing periods

A missing row is not the same as a row containing a missing value. For a regularly sampled series, a timestamp may be absent entirely. For event data, a period with no events may be a true zero—or may reflect a collection outage. Domain knowledge is needed.

```
# Example: hourly sensor data, after selecting one entity.
series = one["measurement"].sort_index()

expected_index = pd.date_range(
    start=series.index.min(),
    end=series.index.max(),
    freq="h",
    tz=series.index.tz
)
coverage = series.reindex(expected_index)

missing_timestamps = coverage.index[coverage.isna()]
missing_periods_n = len(missing_timestamps)
```

Reindexing reveals absent timestamps as missing values; it does not establish why they are absent. Do not fill them with zero unless zero is the correct domain interpretation.

6.1 Coverage metrics

```
coverage_summary = {
    "first_timestamp": series.index.min(),
    "last_timestamp": series.index.max(),
    "observed_n": int(series.notna().sum()),
    "expected_n": len(expected_index),
    "coverage_rate": (
        series.reindex(expected_index).notna().mean()
    )
}
```

For event data, use a carefully defined observation window and exposure denominator. A day with zero recorded events may be a true zero only if the system was operating and collecting events throughout that day.

7. Calendar features and temporal grouping

Calendar summaries can reveal differences by hour, weekday, month, quarter, or season. They are descriptive comparisons; they do not automatically isolate causal calendar effects.

```
ts = df["timestamp"]

df["date"] = ts.dt.date
df["hour"] = ts.dt.hour
df["day_of_week"] = ts.dt.day_name()
```

```
df["month"] = ts.dt.month
df["quarter"] = ts.dt.quarter
df["is_weekend"] = ts.dt.dayofweek >= 5

hour_profile = df.groupby("hour")["measurement"].agg(
    ["count", "mean", "median", "std"]
)
weekday_profile = df.groupby("day_of_week")["measurement"].agg(
    ["count", "mean", "median"]
)
```

Calendar fields must use the timezone appropriate to the question. For example, customer behavior may follow local time even when timestamps are stored in UTC.

8. Trend, seasonality, cycles, and irregular variation

Component	Meaning	Example
Trend	Longer-term direction or level change	Sales rising over several years
Seasonality	Pattern repeating at a known calendar period	Higher retail activity every December
Cycle	Longer or less fixed recurring movement	Business cycle
Irregular variation	Residual short-term movement	Unexpected daily fluctuation
Structural break	Change in the process generating the series	New pricing policy changes the level

These components are conceptual descriptions, not guaranteed separable pieces. A series can have trend and seasonality together, and apparent seasonality may be caused by changing coverage or a calendar-related collection process.

8.1 Plot the raw series first

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(series.index, series.values, linewidth=1)
ax.set_title("Observed time series")
ax.set_xlabel("Time")
ax.set_ylabel("Measurement (unit)")
ax.grid(alpha=0.25)
fig.autofmt_xdate()
fig.tight_layout()
```

Start with the raw time plot. It reveals gaps, spikes, level shifts, changing spread, and possible recurring patterns. Use a clear time axis and label the unit.

9. Resampling and temporal aggregation

Resampling groups observations into time bins, such as hours, days, or months. The aggregation function must match the meaning of the data.

Data meaning	Possible aggregation	Question
Transaction events	Count or sum	How many orders / how much revenue per day?
Instantaneous sensor readings	Mean, median, min, max, quantiles	What was typical or extreme during the hour?
Cumulative counter	Last value, then difference	How much accumulated during the interval?
Rates	Aggregate numerator and denominator, then divide	What was the overall rate?

```
# Event count per day
daily_count = event_df.resample("D", on="event_time").size()

# Daily revenue total
daily_revenue = (
    event_df.set_index("event_time")["revenue"]
            .resample("D").sum(min_count=1)
)

# Hourly sensor summaries
hourly = series.resample("h").agg(
    ["count", "mean", "median", "min", "max"]
)
```

A mean of daily rates is not generally equal to the overall rate. For a ratio, aggregate the numerator and denominator separately, then divide. A sum of cumulative meter readings is usually meaningless; differences between readings may be the relevant quantity.

9.1 Bin boundaries and labels

Resampling conventions such as left/right closed intervals, labels, and calendar anchors affect which bin receives a boundary observation. State these conventions when they matter, especially for financial, operational, or daily reporting.

10. Rolling summaries and changing behavior

A rolling statistic summarizes a moving window. It can help reveal local level and variability changes. A rolling mean is not a forecast; it is a descriptive smoother.

```
rolling_mean = series.rolling(
    window=24, min_periods=18
).mean()

rolling_std = series.rolling(
    window=24, min_periods=18
).std()

fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(series.index, series, alpha=0.45, label="Observed")
ax.plot(rolling_mean.index, rolling_mean,
        linewidth=2, label="24-observation rolling mean")
ax.legend()
ax.set_title("Local level and observed values")
fig.tight_layout()
```

A window of 24 observations means 24 rows, not necessarily 24 hours. For irregularly sampled data, use a time-based window (for example, '24h') when that is the intended interpretation. Rolling calculations can also be affected by gaps and edge windows.

11. Lag, autocovariance, and autocorrelation

A lag compares a series with earlier values. For a regularly sampled series, lag k compares y■ with y■■■. Autocorrelation describes linear association between a series and its lagged version.

For a stationary series with mean μ and variance γ■, the lag-k autocovariance is commonly written γ■ = Cov(Y■, Y■■■), and autocorrelation is ρ■ = γ■/γ■. These summaries depend on the time-series assumptions and sampling structure.

```
# Lag values are meaningful only after checking cadence.
lag_1 = series.shift(1)
lag_24 = series.shift(24)

lag_table = pd.concat(
    [series.rename("y"), lag_1.rename("lag_1"),
     lag_24.rename("lag_24")],
    axis=1
)

lag_correlations = lag_table.corr()
```

A lag of 24 means 24 observations. It represents 24 hours only for an hourly, regular series with no missing intervals after the chosen alignment.

11.1 Autocorrelation function (ACF)

An ACF plot displays estimated autocorrelation across multiple lags. Strong autocorrelation is common in time series and violates the intuition that rows are independent. ACF patterns can reflect trend, seasonality, or short-term persistence; they do not identify a cause by themselves.

```
from statsmodels.graphics.tsaplots import plot_acf

fig, ax = plt.subplots(figsize=(9, 4))
plot_acf(series.dropna(), lags=48, ax=ax, zero=False)
ax.set_title("Autocorrelation of the observed series")
fig.tight_layout()
```

ACF interpretation is sensitive to gaps, irregular sampling, outliers, and nonstationarity. A slowly decaying ACF can arise from trend. Inspect the time plot and distribution before interpreting it.

12. Temporal dependence and stationarity — descriptive intuition

A weakly stationary series has a stable mean, variance, and autocovariance structure over time (with covariance depending on lag rather than absolute time). Many real-world series are not stationary: their level, spread, or dependence changes.

For EDA, ask whether the level and spread appear stable, whether patterns repeat, and whether relationships differ across periods. Do not mechanically difference or transform the series just to make a plot look stable; first describe what changes and when.

12.1 Compare periods

```
df["period"] = df["timestamp"].dt.to_period("Q")
quarter_summary = df.groupby("period")["measurement"].agg(
    ["count", "mean", "median", "std", "min", "max"]
)
```

A period summary can reveal changes in level or spread but can conceal within-period patterns. Always compare the aggregation with the raw or finer-grained time plot.

13. Seasonality and calendar effects

Seasonality is a pattern that repeats at a known calendar interval. Examples include hour-of-day, day-of-week, monthly, and annual patterns. Inspect it with calendar profiles, seasonal subseries plots, and period-over-period comparisons.

```
# Example: average profile by hour and weekday.
profile = df.pivot_table(
    index="day_of_week",
    columns="hour",
    values="measurement",
    aggfunc="median"
)

fig, ax = plt.subplots(figsize=(11, 4))
sns.heatmap(profile, cmap="viridis", ax=ax)
ax.set_title("Median measurement by weekday and hour")
fig.tight_layout()
```

Use consistent definitions of local time and weekday. A heatmap of means may be dominated by a few extreme observations; compare medians or quantiles when appropriate. Also show counts, because some cells may have very few observations.

14. Unusual temporal observations and structural breaks

An isolated spike, a run of unusual values, and a sustained level shift are different patterns. A univariate outlier detector may flag a value that is ordinary given the time of year, or miss a gradual regime change.

• Inspect neighboring observations and the event sequence.

• Compare the same seasonal period across years when sufficient history exists.

• Check source-system outages, sensor maintenance, policy changes, promotions, holidays, and operational incidents.

• Distinguish one-time events from persistent changes.

• Document the detection window, baseline, and uncertainty.

A structural break is a change in the data-generating process, such as a new machine, pricing policy, measurement device, or customer mix. It should not automatically be smoothed away as noise.

15. Multiple time series and panel data

A panel dataset contains repeated observations for multiple entities over time: customers, stores, devices, regions, or products. Analyze both the time dimension and between-entity variation.

```
# Sort and compute within-entity changes.
panel = df.sort_values(["entity_id", "timestamp"]).copy()
panel["previous_value"] = (
    panel.groupby("entity_id")["measurement"].shift(1)
)
panel["change"] = (
    panel["measurement"] - panel["previous_value"]
)

# Summarize coverage by entity.
entity_coverage = panel.groupby("entity_id").agg(
    first_time=("timestamp", "min"),
    last_time=("timestamp", "max"),
    rows=("timestamp", "size"),
    observed_values=("measurement", "count")
```

```
)
```

Do not compare raw levels across entities without considering their scale, operating hours, exposure, or start dates. A store with more recorded transactions may simply be larger or open longer.

16. Common time-series EDA mistakes

• Sorting by timestamp only after calculating lags or rolling values.

• Treating event time and ingestion time as interchangeable.

• Assuming a missing timestamp means a true zero.

• Using row-based lags as if they were fixed-duration lags in irregular data.

• Summing cumulative counters or averaging ratios without checking their meaning.

• Ignoring timezone and daylight-saving effects.

• Choosing a resampling frequency without explaining the temporal question.

• Interpreting a rolling mean as a forecast.

• Calling every spike an outlier without checking seasonal and operational context.

• Reading autocorrelation as proof of causation or a complete model of temporal dependence.

17. Reusable time-series audit

```
def time_series_audit(
    data, time_col, value_col, entity_col=None
):
    work = data.copy()
    work[time_col] = pd.to_datetime(
        work[time_col], errors="coerce", utc=True
    )
    work = work.dropna(subset=[time_col])
    sort_cols = (
        [entity_col, time_col] if entity_col else [time_col]
    )
    work = work.sort_values(sort_cols)

    result = {
        "rows": len(work),
        "time_min": work[time_col].min(),
        "time_max": work[time_col].max(),
        "missing_value_n": int(work[value_col].isna().sum()),
        "duplicate_timestamps_n": int(
            work.duplicated(sort_cols, keep=False).sum()
        )
    }

    if entity_col:
        result["entity_count"] = work[entity_col].nunique()
        result["rows_per_entity"] = (
            work.groupby(entity_col).size().describe()
        )
    else:
        gaps = work[time_col].diff().dropna()
        result["gap_summary"] = gaps.describe()

    return result

# report = time_series_audit(
#     df, "timestamp", "measurement", "entity_id"
# )
```

This function provides an initial audit. It does not infer the correct cadence, decide whether gaps are errors, or resolve timezone semantics. Those require domain context.

18. Summary

• Define what each timestamp represents and the temporal grain of each row.

• Check parsing, timezone, sorting, duplicate timestamps, cadence, and coverage.

• Distinguish absent periods from periods with zero events or missing measurements.

• Explore trend, seasonality, cycles, irregular variation, and structural breaks.

• Choose resampling and aggregation rules that match the measurement meaning.

• Use rolling summaries and lag/ACF diagnostics with sampling assumptions in mind.

• Compare entities and periods without ignoring exposure, scale, or collection differences.

• Treat unusual periods as investigation candidates, not automatic errors.

19. Assignment

Part A — Concepts

• Explain the difference between event time, observation time, and ingestion time.

• Why does a lag of 24 not always mean 24 hours?

• Distinguish trend, seasonality, cycle, and structural break.

• Why is a missing daily row not automatically a zero?

• Explain why the mean of daily rates may differ from the overall rate.

Part B — Temporal audit

Choose a time-indexed dataset. Document its row grain, timezone, expected cadence, and whether it is regular or irregular. Report:

• First and last timestamps, number of observations, and duplicate time keys.

• Timestamp parsing failures and timezone assumptions.

• Observed gap distribution and missing expected periods, if a cadence exists.

• Missing values by variable and missingness by time period.

• Coverage by entity if the dataset is panel data.

Part C — Temporal structure

• Plot the raw series and describe visible level, spread, gaps, spikes, and possible breaks.

• Create at least one calendar profile (hour, weekday, month, or another relevant period).

• Compare at least two aggregation frequencies and explain what each reveals or hides.

• Compute a rolling summary and explain the window definition.

• Inspect lag relationships or an ACF only after checking cadence and missing periods.

Part D — Interpretation

Identify one unusual period. Propose at least three possible explanations and describe what evidence would distinguish them. Do not label it an error without evidence.

20. Coding project — Time Series EDA Workbench

Extend your cumulative EDA master notebook. Use a dataset with a meaningful time index; panel data is welcome but not required.

• Document the observational unit, timestamp semantics, timezone, temporal grain, and expected cadence.

• Build a timestamp audit: parse failures, duplicates, sorting, gap distribution, and temporal coverage.

• Distinguish missing measurements, absent timestamps, and genuine zero-event periods.

• Create raw time plots and calendar summaries appropriate to the domain.

• Compare carefully selected resampling frequencies using justified aggregation functions.

• Add rolling summaries with explicit row-based or time-based window definitions.

• Inspect lag relationships or autocorrelation where the sampling assumptions are reasonable.

• Investigate one unusual period or possible structural break with supporting context.

• Write a report that separates observed temporal patterns from possible explanations.

Acceptance criteria: timestamps are interpreted explicitly; the series is sorted correctly; gaps are not silently converted to zero; aggregation is justified; plots show units and time context; and conclusions acknowledge uncertainty.

Suggested deliverables: updated master notebook, temporal audit table, selected figures, and a concise Markdown report suitable for a professional EDA handoff.

21. Preview of Next Chapter

Chapter 20 — Sampling, Missingness Patterns, and Data Collection Bias

We will examine how data enters a dataset and how the collection process shapes what we can observe. Topics include sampling frames, selection mechanisms, response and nonresponse, coverage gaps, missing-data mechanisms, survivorship and measurement bias, and the limits of conclusions drawn from non-representative data.

Boundary: this chapter studies the evidence and limitations of the observed data. It does not turn into a full causal-inference or survey-weighting course.